

JOBSHEET 7
LAPORAN PERCOBAAN PRAKTIKUM
SEARCHING

Nama : Jonathan Abdiel Haryono
Absen : 14
Kelas : TI 1G

1. Praktikum 1 : Pencarian Menggunakan Algoritma *Sequential Search*

Mahasiswa14.java

```
package Praktikum07;

public class Mahasiswa14 {
    String nim, nama, kelas;
    double ipk;

    Mahasiswa14() {

    }

    Mahasiswa14(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("NIM    : " + nim);
        System.out.println("Nama  : " + nama);
        System.out.println("Kelas : " + kelas);
        System.out.println("IPK   : " + ipk);
    }
}
```

MahasiswaBerprestasi14.java

```
package Praktikum07;

public class MahasiswaBerprestasi14 {

    Mahasiswa14[] listMhs = new Mahasiswa14[5];
    int idx;

    void tambah(Mahasiswa14 mhs) {
        if (idx < listMhs.length) {
            listMhs[idx] = mhs;
        }
    }
}
```

```

        idx++;
    }
    else {
        System.out.println("Data sudah penuh");
    }
}

void tampil() {

    System.out.println("\n-----");
    System.out.println("Menampilkan data seluruh mahasiswa");
    System.out.println("-----");
    for (int i = 0; i < idx; i++) {
        System.out.println("Data Mahasiswa ke-" + (i + 1));
        listMhs[i].tampilInformasi();
        System.out.println("-----");
    }
}

int sequentialSearching(double cari) {
    int posisi = -1;
    for (int j = 0; j < listMhs.length; j++) {
        if (listMhs[j].ipk == cari) {
            posisi = j;
            break;
        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos) {
    if (pos != -1) {
        System.out.println("Data mahasiswa dengan IPK " + x + " ditemukan pada indeks " +
pos);
    }
    else {
        System.out.println("\nData " + x + " tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos) {
    if (pos != -1) {
        System.out.println("NIM\t : " + listMhs[pos].nim);
        System.out.println("Nama\t : " + listMhs[pos].nama);
        System.out.println("Kelas\t : " + listMhs[pos].kelas);
        System.out.println("IPK\t : " + x);
    }
    else {
        System.out.println("\nData mahasiswa dengan IPK " + x + " tidak ditemukan");
    }
}
}

```

MahasiswaDemo14.java

```
package Praktikum07;
import java.util.Scanner;

public class MahasiswaDemo14 {
    public static void main(String[] args) {
        MahasiswaBerprestasi14 list = new MahasiswaBerprestasi14();

        Scanner sc = new Scanner(System.in);

        int jumMhs = 5;

        for (int i = 0; i < jumMhs; i++) {
            System.out.println("Masukkan Data Mahasiswa ke-" + (i + 1));

            System.out.print("NIM    : ");
            String nim = sc.nextLine();

            System.out.print("Nama    : ");
            String nama = sc.nextLine();

            System.out.print("Kelas  : ");
            String kelas = sc.nextLine();

            System.out.print("IPK    : ");
            String ip = sc.nextLine();
            Double ipk = Double.parseDouble(ip);

            System.out.println("-----");
            list.tambah(new Mahasiswa14(nim, nama, kelas, ipk));
        }

        list.tampil();

        System.out.println("\n-----");
        System.out.println("Pencarian data");
        System.out.println("-----");
        System.out.println("\nIPK mahasiswa yang dicari");
        System.out.print("Masukkan IPK : ");
        double cari = sc.nextDouble();

        System.out.println("\nMenggunakan sequential searching");
        double posisi = list.sequentialSearching(cari);
        int pss = (int)posisi;
        list.tampilPosisi(cari, pss);
        list.tampilDataSearch(cari, pss);
    }
}
```

Verifikasi Hasil Percobaan

```
Masukkan Data Mahasiswa ke-1
NIM      : 111
Nama     : Adi
Kelas   : 2
IPK      : 3.6
-----
Masukkan Data Mahasiswa ke-2
NIM      : 222
Nama     : Tio
Kelas   : 2
IPK      : 3.8
-----
Masukkan Data Mahasiswa ke-3
NIM      : 333
Nama     : Ila
Kelas   : 2
IPK      : 3.0
-----
Masukkan Data Mahasiswa ke-4
NIM      : 444
Nama     : Lia
Kelas   : 2
IPK      : 3.5
-----
Masukkan Data Mahasiswa ke-5
NIM      : 555
Nama     : Fia
Kelas   : 2
IPK      : 3.3
-----

-----
Menampilkan data seluruh mahasiswa
-----
Data Mahasiswa ke-1
NIM      : 111
Nama     : Adi
Kelas   : 2
IPK      : 3.6
-----
Data Mahasiswa ke-2
NIM      : 222
Nama     : Tio
Kelas   : 2
IPK      : 3.8
-----
Data Mahasiswa ke-3
NIM      : 333
Nama     : Ila
Kelas   : 2
IPK      : 3.0
-----
Data Mahasiswa ke-4
NIM      : 444
Nama     : Lia
Kelas   : 2
IPK      : 3.5
-----
Data Mahasiswa ke-5
NIM      : 555
Nama     : Fia
Kelas   : 2
IPK      : 3.3
-----
```

```
-----
Pencarian data
-----

IPK mahasiswa yang dicari
Masukkan IPK : 3,5

Menggunakan sequential searching
Data mahasiswa dengan IPK 3.5 ditemukan pada indeks 3
NIM      : 444
Nama     : Lia
Kelas   : 2
IPK      : 3.5
```

Pertanyaan

1. Jelaskan perbedaan metod `tampilDataSearch` dan `tampilPosisi` pada class **MahasiswaBerprestasi**!
2. Jelaskan fungsi `break` pada kode program di bawah ini!

```
if (listMhs[j].ipk==cari){
    posisi=j;
    break;
}
```

3. Apa fungsi variabel `pos` atau indeks hasil pencarian dalam program sequential search?
4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.
5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah `break` dihapus dari kode di atas?

Jawaban

1. Method **tampilPosisi** hanya menampilkan apakah data yang dicari ada atau tidak dan indeks dari data tersebut. Sedangkan, method **tampilDataSearch** menampilkan isi dari data pada indeks yang ditemukan tadi, mulai dari NIM, Nama, hingga IPK-nya.
2. Fungsi **break** pada kode tersebut adalah menghentikan perulangan pencarian jika ditemukan nilai IPK yang sama dengan nilai yang dicari agar perulangan tidak berlanjut karena memang tujuannya hanya menemukan ada atau tidaknya nilai yang dicari mulai dari data yang paling awal hingga akhir dan menampilkan isi datanya.
3. Variabel **pos** atau indeks hasil pencarian dalam program sequential search berfungsi untuk melakukan pengecekan apakah nilainya adalah -1 atau bukan. Jika nilainya bukan -1, artinya data ditemukan, jika nilainya -1, artinya data tidak ditemukan.
4. Tentu yang pertama, karena perulangan pencarian dimulai dari awal hingga akhir ditandai dengan kode program **for (int j = 0; j < listMhs.length; j++)** yang artinya nilai j dari 0 akan terus bertambah seiring berjalannya perulangan. Kemudian, sekiranya nilai sudah ditemukan, perulangan akan langsung berakhir karena adanya **break** dan langsung menampilkan indeks serta isi datanya.
5. Jika perintah **break** dihapus, perulangan searching akan terus melakukan pengecekan bahkan meskipun nilai sudah ditemukan hingga **j < listMhs.length**. Akibatnya, jika terdapat lebih dari satu nilai pada data yang sama dengan nilai yang dicari, program akan memberikan output bukan nilai yang awal, melainkan nilai terakhir dari data-data yang sama tadi. Hal ini akan memperlambat program karena, jika berasumsi bahwa program hanya bertujuan untuk mencari apakah nilai tersebut ada atau tidak dan menampilkan salah satu data yang nilainya sesuai, penghapusan **break** akan membuat program terus berjalan, sehingga dinilai tidak efisien.

2. Praktikum 2 : Pencarian Menggunakan Algoritma *Binary Search*

Mahasiswa14.java

```
package Praktikum07;

public class Mahasiswa14 {
    String nim, nama, kelas;
    double ipk;

    Mahasiswa14() {

    }

    Mahasiswa14(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("NIM    : " + nim);
        System.out.println("Nama   : " + nama);
        System.out.println("Kelas : " + kelas);
        System.out.println("IPK    : " + ipk);
    }
}
```

MahasiswaBerprestasi14.java

```
package Praktikum07;

public class MahasiswaBerprestasi14 {

    Mahasiswa14[] listMhs = new Mahasiswa14[5];
    int idx;

    void tambah(Mahasiswa14 mhs) {
        if (idx < listMhs.length) {
            listMhs[idx] = mhs;
            idx++;
        }
        else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
```

```

        System.out.println("\n-----");
        System.out.println("Menampilkan data seluruh mahasiswa");
        System.out.println("-----");
        for (int i = 0; i < idx; i++) {
            System.out.println("Data Mahasiswa ke-" + (i + 1));
            listMhs[i].tampilInformasi();
            System.out.println("-----");
        }
    }

    int sequentialSearching(double cari) {
        int posisi = -1;
        for (int j = 0; j < listMhs.length; j++) {
            if (listMhs[j].ipk == cari) {
                posisi = j;
                break;
            }
        }
        return posisi;
    }

    int findBinarySearch(double cari, int left, int right) {
        int mid;

        if (right >= left) {
            mid = (left + right) / 2;

            if (cari == listMhs[mid].ipk) {
                return mid;
            }
            else if (listMhs[mid].ipk > cari) {
                return findBinarySearch(cari, left, mid-1);
            }
            else {
                return findBinarySearch(cari, mid+1, right);
            }
        }
        return -1;
    }

    void tampilPosisi(double x, int pos) {
        if (pos != -1) {
            System.out.println("Data mahasiswa dengan IPK " + x + " ditemukan pada indeks " + pos);
        }
        else {
            System.out.println("\nData " + x + " tidak ditemukan");
        }
    }
}

```

```

void tampilDataSearch(double x, int pos) {
    if (pos != -1) {
        System.out.println("NIM\t : " + listMhs[pos].nim);
        System.out.println("Nama\t : " + listMhs[pos].nama);
        System.out.println("Kelas\t : " + listMhs[pos].kelas);
        System.out.println("IPK\t : " + x);
    }
    else {
        System.out.println("\nData mahasiswa dengan IPK " + x + " tidak
ditemukan");
    }
}
}
}

```

MahasiswaDemo14.java

```

package Praktikum07;
import java.util.Scanner;

public class MahasiswaDemo14 {
    public static void main(String[] args) {
        MahasiswaBerprestasi14 list = new MahasiswaBerprestasi14();

        Scanner sc = new Scanner(System.in);

        int jumMhs = 5;

        for (int i = 0; i < jumMhs; i++) {
            System.out.println("Masukkan Data Mahasiswa ke-" + (i + 1));

            System.out.print("NIM    : ");
            String nim = sc.nextLine();

            System.out.print("Nama  : ");
            String nama = sc.nextLine();

            System.out.print("Kelas : ");
            String kelas = sc.nextLine();

            System.out.print("IPK   : ");
            String ip = sc.nextLine();
            Double ipk = Double.parseDouble(ip);

            System.out.println("-----");
            list.tambah(new Mahasiswa14(nim, nama, kelas, ipk));
        }

        list.tampil();
    }
}

```



```

        System.out.println("\n-----");
        System.out.println("Pencarian data");
        System.out.println("-----");
        System.out.println("\nIPK mahasiswa yang dicari");
        System.out.print("Masukkan IPK : ");
        double cari = sc.nextDouble();

        System.out.println("\n-----");
        System.out.println("Menggunakan sequential searching");
        double posisi = list.sequentialSearching(cari);
        int pss = (int)posisi;
        list.tampilPosisi(cari, pss);
        list.tampilDataSearch(cari, pss);

        // MELAKUKAN PENCARIAN DATA BINARY
        System.out.println("\n-----");
        System.out.println("Menggunakan binary searching");
        double posisi2 = list.findBinarySearch(cari, 0, jumMhs-1);
        int pss2 = (int)posisi2;
        list.tampilPosisi(cari, pss2);
        list.tampilDataSearch(cari, pss2);
    }
}

```

Verifikasi Hasil Percobaan

Masukkan Data Mahasiswa ke-1	Menampilkan data seluruh mahasiswa
NIM : 111	-----
Nama : Adi	Data Mahasiswa ke-1
Kelas : 2	NIM : 111
IPK : 3.1	Nama : Adi
-----	Kelas : 2
Masukkan Data Mahasiswa ke-2	IPK : 3.1
NIM : 222	-----
Nama : Ila	Data Mahasiswa ke-2
Kelas : 2	NIM : 222
IPK : 3.2	Nama : Ila
-----	Kelas : 2
Masukkan Data Mahasiswa ke-3	IPK : 3.2
NIM : 333	-----
Nama : Lia	Data Mahasiswa ke-3
Kelas : 2	NIM : 333
IPK : 3.3	Nama : Lia
-----	Kelas : 2
Masukkan Data Mahasiswa ke-4	IPK : 3.3
NIM : 444	-----
Nama : Susi	Data Mahasiswa ke-4
Kelas : 2	NIM : 444
IPK : 3.5	Nama : Susi
-----	Kelas : 2
Masukkan Data Mahasiswa ke-5	IPK : 3.5
NIM : 555	-----
Nama : Anita	Data Mahasiswa ke-5
Kelas : 2	NIM : 555
IPK : 3.7	Nama : Anita
-----	Kelas : 2
	IPK : 3.7

```

-----
Pencarian data
-----

IPK mahasiswa yang dicari
Masukkan IPK : 3,7

-----
Menggunakan sequential searching
Data mahasiswa dengan IPK 3.7 ditemukan pada indeks 4
NIM      : 555
Nama     : Anita
Kelas   : 2
IPK      : 3.7

-----
Menggunakan binary searching
Data mahasiswa dengan IPK 3.7 ditemukan pada indeks 4
NIM      : 555
Nama     : Anita
Kelas   : 2
IPK      : 3.7

```

Pertanyaan

1. Tunjukkan pada kode program yang mana proses *divide* dijalankan!
2. Tunjukkan pada kode program yang mana proses *conquer* dijalankan!
3. Apa fungsi `left`, `right`, dan `mid`?
4. Jika data IPK yang dimasukkan tidak urut. Apakah program masih dapat berjalan? Mengapa demikian?
5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program *binary search* agar hasilnya sesuai
6. Jelaskan bagaimana *binary search* menentukan bahwa data yang dicari tidak ditemukan di dalam array.
7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.

Jawaban

1. Proses *divide* artinya membagi masalah menjadi beberapa bagian masalah. Dijalankan pada kode program ini.

```
mid = (left + right) / 2;
```

2. Proses *conquer* artinya memecahkan masing-masing masalah secara rekursif. Dijalankan pada kode program ini.

```

if (cari == listMhs[mid].ipk) {
    return mid;
}
else if (listMhs[mid].ipk > cari) {
    return findBinarySearch(cari, left, mid-1);
}
else {
    return findBinarySearch(cari, mid+1, right);
}

```

3. Left, right, dan mid digunakan sebagai pointer yang akan menunjuk pada index data.
4. Tentu masih dapat berjalan, namun berjalan tidak sesuai dengan keinginan, bahkan ada kemungkinan untuk tidak menemukan data yang dicari meskipun sebenarnya ada.

Program tetap berjalan karena memang tidak ada batasan atau larangan dalam program untuk melakukan *binary search* pada data yang tidak ter-sorted, sehingga tidak ada yang menyebabkan error ketika program berjalan.

5. Hasil dari *binary search* tidak sesuai karena kesalahan dalam penggunaan *operator assignment*.

Masukkan Data Mahasiswa ke-1	Menampilkan data seluruh mahasiswa
NIM : 111	Data Mahasiswa ke-1
Nama : Adi	NIM : 111
Kelas : 2	Nama : Adi
IPK : 3.8	Kelas : 2
	IPK : 3.8
Masukkan Data Mahasiswa ke-2	
NIM : 222	Data Mahasiswa ke-2
Nama : Ila	NIM : 222
Kelas : 2	Nama : Ila
IPK : 3.7	Kelas : 2
	IPK : 3.7
Masukkan Data Mahasiswa ke-3	
NIM : 333	Data Mahasiswa ke-3
Nama : Lia	NIM : 333
Kelas : 2	Nama : Lia
IPK : 3.5	Kelas : 2
	IPK : 3.5
Masukkan Data Mahasiswa ke-4	
NIM : 444	Data Mahasiswa ke-4
Nama : Susi	NIM : 444
Kelas : 2	Nama : Susi
IPK : 3.4	Kelas : 2
	IPK : 3.4
Masukkan Data Mahasiswa ke-5	
NIM : 555	Data Mahasiswa ke-5
Nama : Anita	NIM : 555
Kelas : 2	Nama : Anita
IPK : 3.2	Kelas : 2
	IPK : 3.2

```

Pencarian data
-----
IPK mahasiswa yang dicari
Masukkan IPK : 3,2
-----
Menggunakan sequential searching
Data mahasiswa dengan IPK 3.2 ditemukan pada indeks 4
NIM : 555
Nama : Anita
Kelas : 2
IPK : 3.2
-----
Menggunakan binary searching
Data 3.2 tidak ditemukan
Data mahasiswa dengan IPK 3.2 tidak ditemukan

```

Perubahan kode ada pada tanda lebih dari menjadi kurang dari.

```

if (right >= left) {
    mid = (left + right) / 2;

    if (cari == listMhs[mid].ipk) {
        return mid;
    }
    else if (listMhs[mid].ipk > cari) {
        return findBinarySearch(cari, left, mid-1);
    }
    else {
        return findBinarySearch(cari, mid+1, right);
    }
}

```



```

if (right >= left) {
    mid = (left + right) / 2;

    if (cari == listMhs[mid].ipk) {
        return mid;
    }
    else if (listMhs[mid].ipk < cari) {
        return findBinarySearch(cari, left, mid-1);
    }
    else {
        return findBinarySearch(cari, mid+1, right);
    }
}

```

Setelah perubahan kode :

```
Masukkan Data Mahasiswa ke-1
NIM : 111
Nama : Adi
Kelas : 2
IPK : 3.8
-----
Masukkan Data Mahasiswa ke-2
NIM : 222
Nama : Ila
Kelas : 2
IPK : 3.7
-----
Masukkan Data Mahasiswa ke-3
NIM : 333
Nama : Lia
Kelas : 2
IPK : 3.5
-----
Masukkan Data Mahasiswa ke-4
NIM : 444
Nama : Susi
Kelas : 2
IPK : 3.4
-----
Masukkan Data Mahasiswa ke-5
NIM : 555
Nama : Anita
Kelas : 2
IPK : 3.2
-----

Menampilkan data seluruh mahasiswa
-----
Data Mahasiswa ke-1
NIM : 111
Nama : Adi
Kelas : 2
IPK : 3.8
-----
Data Mahasiswa ke-2
NIM : 222
Nama : Ila
Kelas : 2
IPK : 3.7
-----
Data Mahasiswa ke-3
NIM : 333
Nama : Lia
Kelas : 2
IPK : 3.5
-----
Data Mahasiswa ke-4
NIM : 444
Nama : Susi
Kelas : 2
IPK : 3.4
-----
Data Mahasiswa ke-5
NIM : 555
Nama : Anita
Kelas : 2
IPK : 3.2
-----

Pencarian data
-----
IPK mahasiswa yang dicari
Masukkan IPK : 3,2
-----
Menggunakan sequential searching
Data mahasiswa dengan IPK 3.2 ditemukan pada indeks 4
NIM : 555
Nama : Anita
Kelas : 2
IPK : 3.2
-----
Menggunakan binary searching
Data mahasiswa dengan IPK 3.2 ditemukan pada indeks 4
NIM : 555
Nama : Anita
Kelas : 2
IPK : 3.2
-----
```

6. *Binary search* menentukan bahwa data yang dicari tidak ditemukan dalam array ketika pointer left sudah lebih dari pointer right.

7.

```
Masukkan jumlah mahasiswa : 3

Masukkan Data Mahasiswa ke-1
NIM      : a
Nama     : a
Kelas   : a
IPK      : 7
-----

Masukkan Data Mahasiswa ke-2
NIM      : b
Nama     : b
Kelas   : b
IPK      : 5
-----

Masukkan Data Mahasiswa ke-3
NIM      : c
Nama     : c
Kelas   : c
IPK      : 2
-----

Menampilkan data seluruh mahasiswa
-----

Data Mahasiswa ke-1
NIM      : a
Nama     : a
Kelas   : a
IPK      : 7.0
-----

Data Mahasiswa ke-2
NIM      : b
Nama     : b
Kelas   : b
IPK      : 5.0
-----

Data Mahasiswa ke-3
NIM      : c
Nama     : c
Kelas   : c
IPK      : 2.0
-----
```

```
-----
Pencarian data
-----

IPK mahasiswa yang dicari
Masukkan IPK : 2

-----

Menggunakan sequential searching
Data mahasiswa dengan IPK 2.0 ditemukan pada indeks 2
NIM      : c
Nama     : c
Kelas   : c
IPK      : 2.0
-----

Menggunakan binary searching
Data mahasiswa dengan IPK 2.0 ditemukan pada indeks 2
NIM      : c
Nama     : c
Kelas   : c
IPK      : 2.0
```

Mahasiswa14.java

```
package Praktikum07;
```

```
public class Mahasiswa14 {
    String nim, nama, kelas;
    double ipk;
```

```
    Mahasiswa14() {

    }
}
```

```
    Mahasiswa14(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
```

```

        kelas = kls;
        ipk = ip;
    }

    void tampilInformasi() {
        System.out.println("NIM    : " + nim);
        System.out.println("Nama   : " + nama);
        System.out.println("Kelas : " + kelas);
        System.out.println("IPK   : " + ipk);
    }
}

```

MahasiswaBerprestasi14.java

```

package Praktikum07;

public class MahasiswaBerprestasi14 {

    Mahasiswa14[] listMhs = new Mahasiswa14[5];
    int idx;

    void tambah(Mahasiswa14 mhs) {
        if (idx < listMhs.length) {
            listMhs[idx] = mhs;
            idx++;
        }
        else {
            System.out.println("Data sudah penuh");
        }
    }

    void tampil() {
        System.out.println("\n-----");
        System.out.println("Menampilkan data seluruh mahasiswa");
        System.out.println("-----");
        for (int i = 0; i < idx; i++) {
            System.out.println("Data Mahasiswa ke-" + (i + 1));
            listMhs[i].tampilInformasi();
            System.out.println("-----");
        }
    }

    int sequentialSearching(double cari) {
        int posisi = -1;
        for (int j = 0; j < listMhs.length; j++) {
            if (listMhs[j].ipk == cari) {
                posisi = j;
                break;
            }
        }
        return posisi;
    }

    int findBinarySearch(double cari, int left, int right) {

```

```

int mid;

if (right >= left) {
    mid = (left + right) / 2;

    if (cari == listMhs[mid].ipk) {
        return mid;
    }
    else if (listMhs[mid].ipk < cari) {
        return findBinarySearch(cari, left, mid-1);
    }
    else {
        return findBinarySearch(cari, mid+1, right);
    }
}
return -1;
}

void tampilPosisi(double x, int pos) {
    if (pos != -1) {
        System.out.println("Data mahasiswa dengan IPK " + x + " ditemukan pada indeks
" + pos);
    }
    else {
        System.out.println("\nData " + x + " tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos) {
    if (pos != -1) {
        System.out.println("NIM\t : " + listMhs[pos].nim);
        System.out.println("Nama\t : " + listMhs[pos].nama);
        System.out.println("Kelas\t : " + listMhs[pos].kelas);
        System.out.println("IPK\t : " + x);
    }
    else {
        System.out.println("\nData mahasiswa dengan IPK " + x + " tidak ditemukan");
    }
}
}
}

```

MahasiswaDemo14.java

```

package Praktikum07;
import java.util.Scanner;

public class MahasiswaDemo14 {
    public static void main(String[] args) {
        // MahasiswaBerprestasi14 list = new MahasiswaBerprestasi14();

        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan jumlah mahasiswa : ");
        int jumMhs = sc.nextInt();
    }
}

```

```

MahasiswaBerprestasi14 list = new MahasiswaBerprestasi14();
Mahasiswa14 mhs[] = new Mahasiswa14[jumMhs];
System.out.println();
sc.nextLine();

for (int i = 0; i < jumMhs; i++) {
    mhs[i] = new Mahasiswa14();
    System.out.println("Masukkan Data Mahasiswa ke-" + (i + 1));

    System.out.print("NIM    : ");
    mhs[i].nim = sc.nextLine();

    System.out.print("Nama    : ");
    mhs[i].nama = sc.nextLine();

    System.out.print("Kelas  : ");
    mhs[i].kelas = sc.nextLine();

    System.out.print("IPK    : ");
    String ip = sc.nextLine();
    mhs[i].ipk = Double.parseDouble(ip);

    System.out.println("-----");

    list.tambah(mhs[i]);
    // list.tambah(new Mahasiswa14(nim, nama, kelas, ipk));
}

list.tampil();

System.out.println("\n-----");
System.out.println("Pencarian data");
System.out.println("-----");
System.out.println("\nIPK mahasiswa yang dicari");
System.out.print("Masukkan IPK : ");
double cari = sc.nextDouble();

System.out.println("\n-----");
System.out.println("Menggunakan sequential searching");
double posisi = list.sequentialSearching(cari);
int pss = (int)posisi;
list.tampilPosisi(cari, pss);
list.tampilDataSearch(cari, pss);

// MELAKUKAN PENCARIAN DATA BINARY
System.out.println("\n-----");
System.out.println("Menggunakan binary searching");
double posisi2 = list.findBinarySearch(cari, 0, jumMhs-1);
int pss2 = (int)posisi2;
list.tampilPosisi(cari, pss2);
list.tampilDataSearch(cari, pss2);
}
}

```